1. Architecture Design

We adopted a **Medallion-lite Architecture** using a **Kimball Dimensional Model**. This hybrid approach ensures data integrity through staging while providing an easy-to-query Star Schema for business users.

Design Justification:

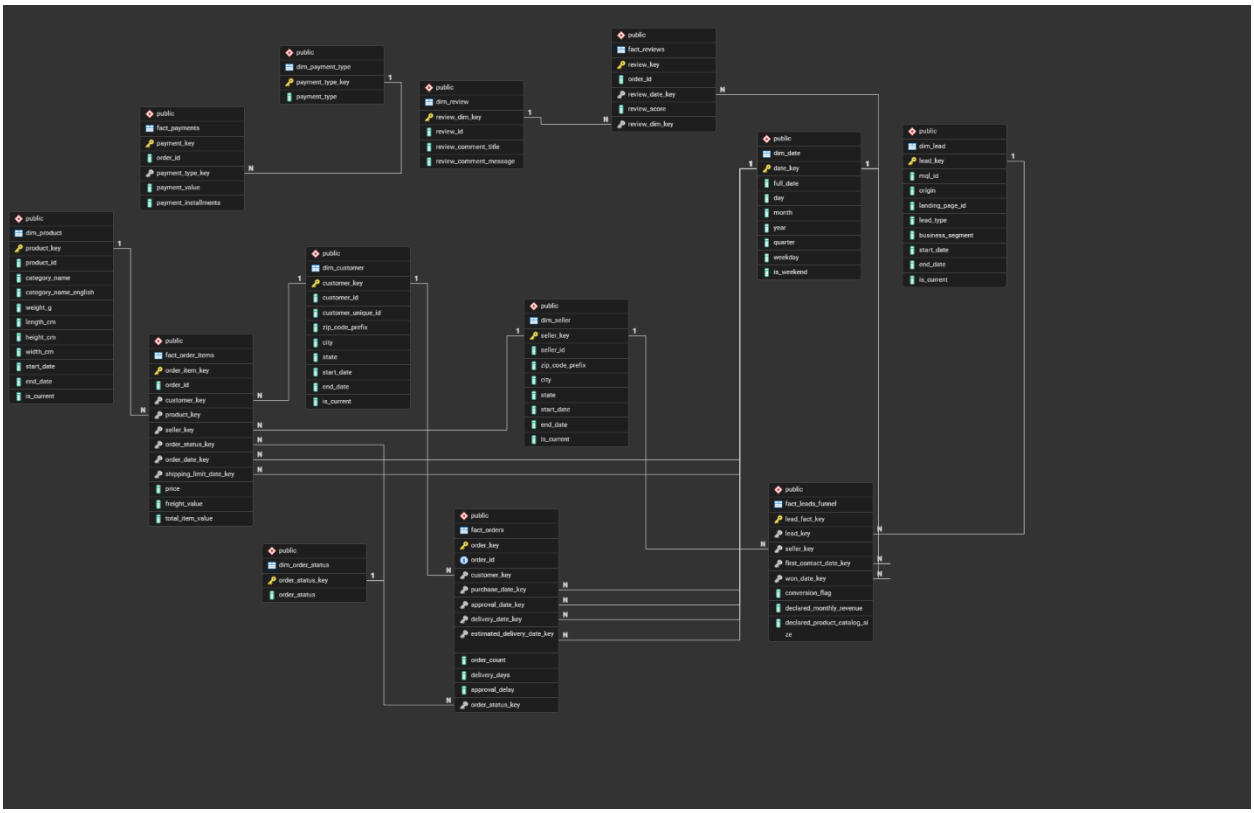
- **Decoupling:** Moving data from SQLite to a Postgres Staging area (brazilian_ecommerce) prevents long-running analytical queries from locking the source system.
- **Kimball Star Schema:** Chosen for the DWH (brazilian_ecommerce_dw) because it optimizes query performance for BI tools and is intuitive for business users.
- **SCD Type 2:** We implemented Slowly Changing Dimensions for Customers, Sellers, and Products to preserve historical accuracy (e.g., tracking a customer's location at the time of purchase).

2. Data Model

The Data Warehouse is structured as a **Star Schema** with centralized Fact tables surrounded by descriptive Dimensions.

Key Entities:

- Fact Tables: fact_orders, fact_order_items, fact_payments, fact_reviews, and fact_leads_funnel. Each represents a distinct business process.
- Dimensions: dim_customer, dim_product, dim_seller, dim_date, and dim_order_status.
- Grain: The grain of fact_order_items is one row per individual item in an order, allowing for deep analysis of product-level revenue.



3. Pipeline Design (ETL)

The pipeline is built using Python and SQLAlchemy, focusing on modularity and fault tolerance.

Pipeline Stages:

1. **E**xtraction: Bulk migration from SQLite to Postgres Staging with chunksize processing to handle large tables within memory limits.

2. Transformation (Dimensions): Logic to detect changes in customer or product attributes. Existing records are "expired" and new versions are inserted to maintain history (SCD Type 2).
3. Loading (Facts): Mapping business IDs to surrogate keys. We handle missing data by using an "Unknown" (-1) record in dim_date to maintain referential integrity without losing transactional data.

4. Performance & Scalability

- **Indexing:** B-Tree indexes are applied to all surrogate keys and order_id columns in the DWH to speed up joins.
- **Batch Processing:** The Python engine uses chunked inserts (5,000 rows per batch) to avoid memory overflows and stay within PostgreSQL's parameter limits.
- **Atomic Transactions:** All transformations are wrapped in database transactions; if a step fails, the warehouse rolls back to its last consistent state.

5. Key Assumptions & Trade-offs

- **Assumption:** The dim_date table is pre-populated for a 10-year range to simplify date-based partitioning.
- **Trade-off:** We chose **Python-based joins** over SQL-based INSERT INTO ... SELECT to keep the logic independent of the database engine, making the pipeline easier to migrate to other cloud providers in the future.

Sample Analytical Queries:

```
-- 1. How are sales trending over time?
SELECT
    d.year,
    d.month,
    SUM(f.price) as total_revenue,
    COUNT(DISTINCT f.order_id) as total_orders
```

```
FROM fact_order_items f
JOIN dim_date d ON f.order_date_key = d.date_key
GROUP BY d.year, d.month
ORDER BY d.year, d.month;

-- 2. Who are the most valuable customers? (Top 10 by Revenue)
SELECT
   c.customer_unique_id,
   c.city,
   SUM(f.price) as total_spent
FROM fact_order_items f
JOIN dim_customer c ON f.customer_key = c.customer_key
GROUP BY c.customer_unique_id, c.city
ORDER BY total_spent DESC
LIMIT 10;

-- 3. What affects delivery performance? (Average delivery days by carrier status)
SELECT
   s.order_status,
   AVG(f.delivery_days) as avg_delivery_time
FROM fact_orders f
JOIN dim_order_status s ON f.order_status_key = s.order_status_key
WHERE f.delivery_days IS NOT NULL
GROUP BY s.order_status;

-- 4. Which product categories drive revenue?
```

```
SELECT
    p.category_name_english,
    SUM(f.price) as revenue
FROM fact_order_items f
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY p.category_name_english
ORDER BY revenue DESC;
```